

LLMProvider

LLMProvider

یک رابط، ۴۳ ارائه‌دهنده — با مدارشکن، تلاش مجدد و سلامت یکپارچه

خلاصه

LLMProvider عمومی و قابل استفاده مجدد است که یک رابط یکپارچه Go یک ماژول به همراه الگوهای تاب‌آوری عملیاتی — مدارشکن، پایش سلامت، تلاش مجدد با LLMProvider تأخیر تصاعدی، بارگذاری تنبل — تعریف می‌کند و ۴۳ پیاده‌سازی ارائه‌دهنده مشخص را پشت این قرارداد و کشف مدل بدون جایگزینی OpenAI واحد ارائه می‌دهد، به همراه یک آداپتور عمومی سازگار با سخت‌کد شده.

توضیح کوتاه

LLMProvider (Complete) قابل استفاده مجدد که یک رابط Go یک ماژول به همراه CompleteStream، HealthCheck، GetCapabilities، ValidateConfig — مدارشکن، پایشگر سلامت، تلاش مجدد با تأخیر تصاعدی، مقداردهی اولیه مکانیزم‌های تحمل خطا — مدارشکن، پایشگر سلامت، تلاش مجدد با تأخیر تصاعدی، مقداردهی اولیه ارائه می‌دهد. این ماژول OpenAI تنبل — روی ۴۳ آداپتور ارائه‌دهنده و یک آداپتور عمومی سازگار با نخ‌نسبت است.

توضیح مفصل

به آن نیاز دارد، اما LLM همان لایه انتزاعی است که هر سرویس مصرف‌کننده LLMProvider تقریباً هیچ‌کس آن را به درستی پیاده‌سازی نمی‌کند — همان لوله‌کشی ناخوشایند که یک دمورا از سیستمی که در برابر ترافیک واقعی حوام می‌آورد، متمایز می‌سازد. این ماژول یک رابط واحد با Complete، CompleteStream، HealthCheck، قابلیت‌های مشخص تعریف می‌کند تا کد برنامه تنها به یک قرارداد واحد — GetCapabilities، ValidateConfig وابسته باشد، فارغ از اینکه کدام یک از ۴۳ بک‌اند پاسخگوی درخواست باشد. سپس، مکانیزم‌های مقاوم‌سازی عملیاتی را ارائه می‌دهد که تماس‌های شکننده با ارائه‌دهندگان را به چیزی تبدیل می‌کند که می‌توان بدون نگرانی در محیط عملیاتی اجرا کرد.

از جمله کانال — یک مدار شکن سه حالته (بسته → باز → نیمه باز) به صورت شفاف هر رائه دهنده ای را در بر می گیرد، جایی که یک استریم خالی به درستی به عنوان شکست در نظر گرفته — استریم آن می شود. بدین ترتیب، یک بک اند ناپایدار می تواند مدار را باز کند و از سقوط کل سرویس جلوگیری نماید؛ مرکزی تمام مدار شکن ها را به صورت همزمان رصد می کند CircuitBreakerManager یک

یک پایشگر سلامت قابل تنظیم، رائه دهنده گان را به طور مداوم در حالت های سالم، تخریب شده، ناسالم، یا ناشناخته بر اساس آستانه ها و بلزدهای زمانی بررسی می کند تا تخریب به جای کشف در زمان قطعی، پیش از وقوع شناسایی شود.

منطق تلاش مجدد، تأخیر تصاعدی را با تأخیر تصادفی ترکیب می کند و تصمیمات هوشمندانه ای بر دوبره (خطاهای گفرا شبکه، ۵، ۴۲۹XX) اساس وضعیت اتخاذ می نماید — خطاهای قابل تلاش مجدد یا زمینه لغو شده، چرخه ای تلف نمی شود. تأخیرها نیز محدود XX امتحان می شوند، اما برای خطاهای ۴ می شوند تا طوفان تلاش مجدد رخ ندهد.

— الگوی مقداردهی اولیه تنبل، ساخت هر رائه دهنده را تا اولین استفاده واقعی به تعویق می اندازد. انتخابی آگاهانه که ثبت تمام ۴۳ رائه دهنده را عملاً بدون هزینه نگه می دارد.

رائه OpenAI سازگار با generic این مازول ۴۳ بسته رائه دهنده مشخص به همراه یک آداپتور پیاده سازی می کند — احراز /v1/chat/completions / نقطه پایانی هر می دهد که رابط کامل را برای به طوری که رائه دهنده ای بدون بسته — [DONE] با مدیریت صحیح SSE استریم، Bearer هویت آن، به عنوان یک شهروند درجه یک در نظر گرفته می شود URL اختصاصی، به محض اشره آداپتور به

با استفاده از قرارداد سخت گیرانه، apikeys) اعتبارنامه ها تنها در یک مکان حل می شوند و بدین ترتیب کل دسته ای از باگ ها که در آن «کلید (<Provider> ApiKey سخت کد شده تست ها را پاس می کند، اما کلید واقعی هرگز متصل نشده و محصول در محیط عملیاتی از کار می افتد» از مبدأ مسدود می شود.

کشف مدل ها به طور عمدی و تقریباً لجوجانه صادقانه انجام می شود: این مازول با استفاده از یک حافظه — های رائه دهنده گان زنده را پرسوجو می کند و — مطابق با سیاست های حاکمیتی API، TTL، نهان لایه جایگزین سخت کد شده قدیمی به کلی حذف شده است. هنگامی که کشف زنده با شکست مواجه بر نمی گرداند تا فراخواننده هرگز شناسه مدلی را دریافت نکند هیچ چیز LLMProvider، می شود که ظاهراً معتبر است اما قابل فراخوانی نیست. تمام این اجزا برای استفاده همزمان طراحی شده و نخ نسبت هستند.

محتوا

چرا آن را ساختیم

در محیط عملیاتی شکست می خورند — رائه دهنده گان محدودیت نرخ اعمال LLM فراخوانی های ساده می کنند، کیفیت خدمات را کاهش می دهند یا از دسترس خارج می شوند، و یک بک اند معیوب می تواند کل سرویس را با خود به زمین بزند. کاتالوگ مدل ها تغییر می کند و فهرست های سخت کد شده توسط LLMProvider، شناسه هایی را در اختیار تماس گیرندگان قرار می دهند که دیگر کار نمی کنند الگوهای تاب آوری و کشف صادقانه را متمرکز می کند تا هر مصرف کننده ای به طور خودکار تحمل خطا و صداقت را به ارث ببرد.

چرا انقلابی است

را به یک حرکت واحد تقلیل می‌دهد — کافی است یک واسط «LLM» این ابزار «ادغام یک رائه‌دهنده پیاده‌سازی کنید یا فقط آدپتور عمومی را به یک نقطه پایانی نشانه بگیرید — و سپس آن رائه‌دهنده را به‌طور خودکار و شفاف در مدارشکن، پایش سلامت و تلاش مجدد با تأخیر تصادفی شده می‌پیچد. تاب‌آوری دیگر چیزی نیست که هر تیمی آن را از نو اختراع کند (بد، تحت فشار ضرب‌العجل، پس از اولین قطعی)، بلکه به رفتار پیش‌فرض کتابخانه در همه ۴۳ بک‌اند تبدیل می‌شود. مهندسی قابلیت اطمینان یک بار نوشته می‌شود، به دقت آزمایش می‌شود و به‌طور رایگان در اختیار هر کسی قرار می‌گیرد که آن را وارد کند.

نوآوری‌ها

- تکمیل، جریان‌سازی، سلامت، قابلیت‌ها و اعتبارسنجی تنظیمات — یک واسط آگاه به قابلیت‌ها در یک قرارداد واحد ادغام شده‌اند که هر بک‌اندی به‌طور یکسان به آن پایبند است.
- مدارشکن نه تنها از کانال **پیچیدن مدارشکن به صورت شفاف** — از جمله جریان‌ها بلکه از کل جریان محافظت می‌کند و یک جریان خالی را به عنوان شکست `CompleteStream` واقعی تلقی می‌کند — با اطلاع‌رسانی به شنوندگان به صورت بدون بن‌بست و خارج از قفل بسته‌های اختصاصی سبک باقی — **OpenAI بسته رائه‌دهنده + یک آدپتور سازگار با ۴۳**، پشتیبانی کند `/v1/chat/completions` می‌مانند و هر فروشنده فهرست‌نشده‌ای که از به محض اینکه آدپتور را به سمت آن نشانه بگیرید، کار خواهد کرد.
- دقیقاً یک مکان متغیرهای محیطی — **(apikeys) مرجع واحد اعتبارنامه** را می‌خواند و به جای هشدار درباره ناسازگاری «آزمایش‌های سبز `ApiKey_<Provider>` محصول معیوب»، آن را به‌طور ساختاری حذف می‌کند.
- های رائه‌دهندگان زنده پشت یک **API — کشف صادقانه مدل (بدون جایگزین سخت‌کد شده)** برمی‌گرداند و هرگز کاتالوگی منسوخ یا `nil`، حافظه پنهان با عمر محدود؛ در صورت شکست ساختگی رائه نمی‌دهد که شناسه‌های غیرقابل فراخوانی توزیع کند.
- ساخت در اولین استفاده به تعویق می‌افتد، بنابراین ثبت همه ۴۳ — **sync.Once** آغاز تنبل با رائه‌دهنده تقریباً هیچ هزینه‌ای ندارد تازمانی که واقعاً یکی از آن‌ها فراخوانی شود.
- یک اجراگر واقعی که رفتار مدارشکن، سلامت و تلاش — **پشته چالش ضد بلوف و چندمنطقه‌ای** مجدداً در پنج منطقه آزمایش می‌کند و با آزمایش جهش جفت‌شده کنترل می‌شود (کد بدون جهش باید با خروجی ۰ پایان یابد؛ یک جهش تزریق‌شده باید خروجی ۹۹ را مجبور کند)، بنابراین یک مجموعه آزمایش موفق به‌طور قطعی به معنای رفتار کرآمد است.

بزرگ‌ترین چالش‌های فنی و راه‌حل‌های ما

- یک بک‌اند ناپایدار نباید کل سرویس را با خود به زمین بزند. شکست‌های آبخاری رائه‌دهندگان این مشکل با یک مدارشکن سه‌حالت (بسته → باز → نیمه‌باز) حل شد که به‌طور شفاف هر رائه‌دهنده‌ای — از جمله جریان آن — را می‌پیچد، در صورت شکست مداوم باز می‌شود، در حالت مرکزی هماهنگ `CircuitBreakerManager` نیمه‌باز برای بازیابی کاوش می‌کند و توسط یک می‌شود.
- با تلاش مجدد نمایی آگاه به وضعیت به علاوه تأخیر تصادفی حل. **خطاهای گزرا و محدودیت نرخ** تا $\min(\text{InitialDelay} \cdot \text{Multiplier}^{(n-1)}, \text{MaxDelay}) \pm \text{jitter}$ — شد تلاش‌های مجدد به جای همگام‌سازی در یک گله شلوغ، پخش شوند. این مکانیزم دقیقاً آنچه را که باید دوباره تلاش کند (خطاهای ۴۲۹، ۵۰۰، ۵۰۲، ۵۰۳، ۵۰۴ و خطاهای شبکه) دوباره امتحان دیگر خودداری می‌کند **XX** می‌کند و از تلاش برای یک زمینه لغوشده یا هر خطای ۴ با ثبت ۴۳ رائه‌دهنده که تنها تعداد کمی. **مقیاس‌بندی برای رائه‌دهندگان ثبت‌شده اما بلااستفاده** از آن‌ها در هر سرویس زنده هستند، ساخت مشتاقانه صرفاً اتلاف منابع خواهد بود. این مشکل با

حل شد، بنابراین تنها ارائه‌دهندگانی که واقعاً sync.Once آغاز تنبل محافظت‌شده توسط فراخوانی می‌شوند هزینه‌راه‌اندازی خود را پرداخت می‌کنند.

- با حذف کامل لایه جایگزین کشف سخت‌کد شده (بر اساس توزیع شناسه‌های مدل نامعتبر و برگرداندن هیچ‌چیز در صورت شکست کشف‌زنده حل شد — به‌علاوه یک CONST-036) کپی تدافعی هنگام برگشت تا تماس‌گیرنده نتواند حافظه پنهان را تغییر دهد یا با خواننده دیگری تداخل کند. صداقت به‌طور ساختاری اعمال می‌شود، نه بر اساس قرارداد.
- حالت شکست ظریف، بن‌بست بین قفل مدارشکن و تماس‌های جریان‌سازی + صحت همزمانی بازگشتی شنوندگان آن است. این مشکل با عکس‌برداری لحظه‌ای از شنوندگان و اطلاع‌رسانی به آن‌ها خارج از قفل و تحت یک مهلت ۵ ثانیه‌ای حل شد، و همچنین با باز کردن قفل پیش از اطلاع‌رسانی در زمان بلز نشانی — به‌علاوه اینکه همه مؤلفه‌ها برای استفاده همزمان ساخته شده و تثبیت شده‌اند race - توسط مجموعه آزمایش

محتوا

پشته فناوری

- انتخاب‌شده به‌خاطر همزمانی درجه‌یک، بایرنی‌های ایستا و کتابخانه — (نسخه ۱.۲۵.۳) **Go** استاندارد قدرتمند؛ شامل ماژول، رابط، همه عناصر تاب‌آوری و هر ۴۳ آداپتور است.
- بدون وابستگی عمدی: موتور کلاینت‌های HTTP — (کتابخانه استاندارد) **net/http** و فراخوانی‌های کشف‌زنده است، بنابراین OpenAI اختصاصی هر ارائه‌دهنده، آداپتور سازگار با نیازی به بررسی یا به‌روزرسانی هیچ حمل‌ونقل شخص ثالث نیست.
- ثبت رویداد ساختاریافته و حساس به سطح، دقیقاً در جاهایی که اپراتورها به دید نیاز — **logrus** دارند: درون تغییر وضعیت قطع‌کننده مدار و مسیر کشف.
- موتور مجموعه‌آزمون و به‌ویژه تثبیت انشعاب جهش‌یافته که باعث می‌شود اجرای — **testify** موفق معنای واقعی داشته باشد.
- فایل‌های بسته‌های چندزبانه و پیکربندی را در قالبی قابل ویرایش توسط انسان — **yaml.v3** تجزیه می‌کند.
- و **digital.vasic.models** LLMRequest، LLMResponse انواع مشترک — همگی در یک مکان نگهداری می‌شوند تا هر آداپتور از واژگان ProviderCapabilities، یکسانی استفاده کند (یک وابستگی زمان اجرا با مستندات)
- **circuit.health.retry.apikeys.discovery.providers/** — بسته‌های داخلی سطح تاب‌آوری و یکپارچه‌سازی به واحدهای کوچک و **il8n** و (generic + فروشنده ۴۳) مستقل قابل آزمون تقسیم شده است، نه یک مجموعه یکپارچه.
- یک منبع واحد و بی‌ابهام — **ApiKey_<Provider>** قرارداد **~/api_keys.sh + .env** برای اعتبارنامه‌ها، به‌گونه‌ای که کلیدها در آزمون‌ها و محیط عملیاتی به یک شکل متصل می‌شوند.
- ستون فقرات — اجراکننده چالش **+ (-race -p 1) Makefile** مجموعه آزمون همزمانی ضدفریب: آشکارساز همزمانی درستی همزمانی را اثبات می‌کند و اجراکننده چالش رفتار واقعی را مقیاس‌بندی، فشار، کشف‌زنده و سناریوهای بدون تعلیق، DDoS تحت هرج‌ومرج، حملات آزمایش می‌کند.

وضعیت و یادداشت‌های شفافیت

- عمومی است GitHub یک ماژول قابل استفاده مجدد و جداشده؛ مخزن وضعیت: بتا.
- را اعلام می‌کند در حالی که MIT مجوز doc.go — ناسازگاری وجود دارد. مجوز: نامشخص.
- است — پیش از انتشار، تأیید کنید Apache-2.0 به سبک LICENSE فایل
- **helix-** منبع واحد بالادستی برای کاتالوگ مدل‌های معتبر است. مانیفست **LLMsVerifier** اعلام کرده در حالی که مستندات **deps: []** قدیمی به‌نظر می‌رسد (وابستگی‌ها را **deps.yaml**

در «(models.dev) را ذکر می‌کنند؛» طبقهٔ ۲ digital.vasic.models وابستگی به بخش کشف، یک طرح اولیه برنامه‌ریزی شده است و فعال نیست.

(زیرساخت — ماژول قابل استفادهٔ مجدد و جداشده-LLM خوشه) اصلی-Helix: طبقهٔ اولویت قرار دارد HelixTrack پس از